

LabscriptAI authoring benchmark — task catalog

Source: benchmarks/authoring/tasks.yaml

Generated: 2026-05-17

Tasks: 90

T001 difficulty=Easy holdout=yes source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

Basic liquid handling with different liquid types:

1. Load a 96-well plate and source tubes
2. Transfer the following liquids (50uL each):
 - Water (low viscosity)
 - Glycerol solution (high viscosity)
 - Ethanol solution (volatile)
3. Adjust pipetting speed and aspiration/dispense rates for each liquid type
4. Add proper mixing for viscous liquids
5. Handle volatile liquid evaporation

T002 difficulty=Easy holdout=yes source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

Basic liquid transfer between wells:

1. Load a 96-well plate with blue dye
2. Transfer 100uL from well A1 to well B1
3. Transfer 50uL from well B1 to well C1
4. Include proper tip handling
5. Complete the protocol

T003 difficulty=Easy holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

Empty well handling:

1. Set up a 96-well plate with some wells empty
2. Attempt transfers from wells A1-H12
3. Handle cases where:
 - Source well is empty (skip with warning)
 - Destination well is already occupied
 - Tip touches dry well surface
4. Include appropriate error messages

T004 difficulty=Easy holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

I want to write a Python script that runs Opentrons machine. This robot can be used to automate laboratory experiment, and is used by many researchers in biology.

Can you write down a script that does the following experiment?

Protocol Steps:

1. Wash the 6 well plate with PBS(-) by pipetting up and down 5 times. For each well, use 1 ml of PBS(-).
2. After washing, add 1 ml of trypsin solution (0.25%) to the cell culture dish and wait for 5 minutes.
3. Then, add 1 mL of cell culture medium (DMEM) to the 6 well plate.
4. Finish the experiment.

Possible Labware and Pipettes (Note that you don't need to trust these items, you can use any labware and pipette you want):

1. opentrons_10_tuberack_falcon_4x50ml_6x15ml_conical
2. p1000_single
3. corning_6_wellplate_16.8ml_flat

When writing scripts, be aware of the following:

1. Do not aspirate more than pipette's max volume. For example, if you want to aspirate 2000 uL with a p1000_single, you need to split it into two commands.
2. Check the number of tips you use to avoid running out of tips.
3. when load_labware('name', slot), you can't declare the same slot twice, you need to use different slots for different labware.

T005 difficulty=Easy holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

Load the following labware:

1. A custom_96_wellplate_200ul (this is an invalid labware name)
2. A special_tube_rack_15ml (this is also invalid)
3. Transfer between them

This test checks labware name validation.

T006 difficulty=Easy holdout=yes source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

Multi-well sample preparation:

1. Add 50uL buffer to wells A1-H8
2. Add 10uL enzyme to same wells
3. Mix by pipetting up and down 2 times
4. Incubate for 5 minutes
5. Add 100uL stop solution to same wells
6. Mix one final time

T007 difficulty=Easy holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

Plate washing protocol:

1. Load 96-well plate with samples
2. Aspirate liquid from all wells (A1-H12)
3. Add 200uL wash buffer to all wells
4. Mix by pipetting up and down once
5. Aspirate wash buffer from all wells
6. Repeat wash step once more

T008 difficulty=Easy holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

Sample dilution basics:

1. Stock solution in tube A1: 1mg/mL
2. Create 1:2 dilution in tube B1 (add 50uL stock + 50uL buffer)
3. Create 1:4 dilution in tube C1 (add 50uL from B1 + 50uL buffer)
4. Mix each tube thoroughly
5. Transfer final dilutions to plate wells

T009 difficulty=Easy holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

Simple mixing protocol:

1. Load a 96-well plate with reagents
2. Add 100uL reagent A to wells A1-A6
3. Add 100uL reagent B to same wells
4. Mix each well by pipetting up and down 3 times
5. Dispose tips properly

T010 difficulty=Easy holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

Simple serial dilution calculation:

1. Prepare serial dilutions from stock solution
2. Stock: 10mg/mL in tube A1
3. Create working solutions:
 - 1mg/mL (1:10 dilution)
 - 0.1mg/mL (1:100 dilution)
 - 0.01mg/mL (1:1000 dilution)
4. Calculate volumes automatically
5. Verify final concentrations in code comments

T011 difficulty=Easy holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

Single channel pipette basics:

1. Use P300 single channel pipette
2. Transfer 200uL from source tube to 6 different wells
3. Use fresh tip for each transfer
4. Track tip usage
5. End with tip disposal

T012 difficulty=Easy holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

Use a P1000 pipette to transfer large volumes:

1. Transfer 1500uL from source tube to destination well
2. Transfer 2000uL from another source to another well
3. Complete the protocol

Pay attention to pipette volume limits.

T013 difficulty=Easy holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

Volume consistency check:

1. Transfer identical volumes to all wells in 96-well plate
2. Use 200uL of blue dye solution
3. Transfer to every well (A1-H12)
4. Include quality check:
 - Verify all wells received correct volume
 - Check for any missed wells
 - Log transfer completion for each well
5. Test pipette accuracy across full plate range

T014 difficulty=Medium holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

Batch processing with quality control:

1. Process 12 samples in parallel:
 - Add 100uL reagent A to each sample
 - Mix 3 times
 - Add 50uL reagent B to each sample
 - Mix 2 times
2. Include quality check after each batch
3. Log processing times and any issues
4. Handle individual sample failures without stopping entire batch

T015 difficulty=Medium holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

Cell culture medium exchange protocol (parameterized):

The robot should handle different cell types (hMSC/iPS) and media combinations:

- Cells are cultured in 6 well plates
- Exchange old medium with PBS(-) wash (2x1ml per well)
- Add new medium (DMEM or SCM130, 1ml per well)

This should work for any combination of cell type and medium.

T016 difficulty=Medium holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

Contamination prevention protocol:

1. Handle different types of liquids:
 - Surfactant-containing solutions
 - High-viscosity solutions
 - Protein solutions
2. Use proper aspiration techniques:
 - Slow aspiration for viscous liquids
 - Pre-wetting tips
 - Avoid air bubble formation
3. Include visual inspection steps in code

T017 difficulty=Medium holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

Cross-contamination prevention:

1. Transfer two different colored dyes:
 - Blue dye: wells A1-A6
 - Red dye: wells B1-B6
2. Use proper tip changing strategy to prevent cross-contamination
3. Add wash steps between different dye transfers
4. Include visual verification in protocol comments
5. Test for color mixing in boundary wells

T018 difficulty=Medium holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

Data logging and reporting:

1. Create a comprehensive transfer protocol
2. Log all transfer details:
 - Source and destination positions
 - Transfer volumes and timestamps
 - Tip usage tracking
 - Error events
3. Generate end-of-run summary report
4. Export data to CSV format for analysis

T019 difficulty=Medium holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

Dynamic liquid level detection:

1. Set up a transfer protocol where source liquid level decreases
2. The robot should:
 - Detect when source well is empty before transfer
 - Automatically switch to next source well
 - Log the well switching events
 - Handle cases where all sources are exhausted
3. Add error handling for insufficient total volume

T020 difficulty=Medium holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

Immunostaining protocol (parameterized for different targets):

Cells are cultured in 6 well plates. The robot should:

1. Fix cells with 4% paraformaldehyde (2ml per well)
2. Wash 3x with PBS(-)
3. Permeabilize with 0.1% Triton X-100 (500ul per well)
4. Block with blocking buffer (1ml per well)
5. Add primary antibody (different targets: lysosomes/E-Cadherin)
6. Wash and add secondary antibody
7. Add mounting medium

Make the protocol flexible for different antibody combinations.

T021 difficulty=Medium holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

Liquid level adaptation:

1. Fill wells in a 96-well plate to different heights:
 - Row A: 50uL (low level)
 - Row B: 200uL (high level)
 - Row C: 100uL (medium level)
2. Test aspiration from different liquid levels
3. Adjust pipette depth for each level
4. Ensure consistent aspiration without air bubbles

T022 difficulty=Medium holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

Load multiple labware items and ensure proper slot management:

1. Load a 96-well plate for samples
2. Load a tube rack for reagents
3. Load a tip rack for pipetting
4. Transfer 100uL from tube A1 to well A1 of the plate
5. Make sure to use different slots for each labware item

This test specifically checks for slot conflict errors.

T023 difficulty=Medium holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

Multi-pipette coordination:

1. Load both P1000 and P300 pipettes
2. Assign appropriate tasks:
 - P1000: Transfer volumes >300uL
 - P300: Transfer volumes <=300uL
3. Coordinate both pipettes in same protocol
4. Manage tip racks for both pipettes
5. Optimize workflow to minimize pipette switching

T024 difficulty=Medium holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

PCR Setup Protocol:

1. Load a 96-well PCR plate
 2. Load reagent tubes containing: Master Mix, Forward Primer, Reverse Primer, Template DNA
 3. Add 10uL Master Mix to each well (A1-H12)
 4. Add 1uL Forward Primer to each well
 5. Add 1uL Reverse Primer to each well
 6. Add 2uL Template DNA to wells A1-A8 only
 7. Mix each well by pipetting up and down 3 times
- Use appropriate pipettes and tip management.

T025 difficulty=Medium holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

Perform a 96-well transfer protocol:

1. Transfer 50uL from each well of source plate to destination plate
 2. Use proper tip management
 3. Handle tip rack capacity limitations
- This requires managing tip usage across multiple transfers.

T026 difficulty=Medium holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

Plate mapping and tracking:

1. Create a sample plate layout with specific pattern:
 - Standards in columns 1-2 (A1-H2)
 - Controls in columns 11-12 (A11-H12)
 - Samples in columns 3-10 (A3-H10)
2. Implement proper well mapping logic
3. Track sample ID to well position mapping
4. Include logging for all transfers
5. Generate plate map summary at end

T027 difficulty=Medium holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

Protocol parameterization:

1. Create a flexible protocol that accepts parameters:
 - Number of samples (variable: 1-96)
 - Transfer volume (variable: 10-500uL)
 - Mixing repetitions (variable: 1-10)
 - Incubation time (variable: 1-60 min)
2. Validate input parameters
3. Adapt protocol behavior based on inputs
4. Include parameter range checking

T028 difficulty=Medium holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

Reagent preparation on-deck:

1. Prepare working solutions from stock reagents:
 - Dilute enzyme stock 1:10 in buffer
 - Mix substrate with diluent
 - Prepare stop solution
2. Use separate tubes for each preparation
3. Include mixing steps for homogeneity
4. Label and track prepared reagents
5. Use freshly prepared reagents in subsequent steps

T029 difficulty=Medium holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

Temperature-sensitive reagent handling:

1. Load a temperature module with 4C cooling
2. Place reagents that need to stay cold:
 - Enzyme solution (50uL)
 - Sensitive substrate (100uL)
3. Keep reagents on temperature module during transfers
4. Transfer cold reagents to assay plate at room temperature

5. Minimize time outside temperature control

T030 difficulty=Medium holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

Time-sensitive reaction sequencing:

1. Set up a time-sensitive reaction with multiple steps:
 - Add reagent A (50uL) and wait exactly 2 minutes
 - Add reagent B (50uL) and wait exactly 30 seconds
 - Add reagent C (100uL) immediately
 - Mix and transfer to detection plate
2. Use protocol.delay() for precise timing
3. Log each step with timestamps

T031 difficulty=Medium holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

Tip usage optimization strategy:

1. Set up protocol with multiple transfer steps
2. Optimize tip usage:
 - Reuse tips for same reagent transfers
 - Change tips only when switching reagents
 - Use tip rack efficiently
3. Process 96-well plate with minimal tip usage
4. Track tip consumption throughout protocol

T032 difficulty=Medium holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

Variable volume calculations:

1. Calculate reagent volumes based on well-specific requirements:
 - Wells A1-A6: 50uL (low concentration samples)
 - Wells B1-B6: 100uL (medium concentration samples)
 - Wells C1-C6: 200uL (high concentration samples)
2. Implement volume calculation logic
3. Adjust for different sample types automatically
4. Include volume verification checks
5. Log calculated vs actual volumes

T033 difficulty=Medium holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

Well format conversion:

1. Transfer samples from 24-well plate to 96-well plate
2. Each 24-well contains 1000uL sample
3. Distribute equally to 4 corresponding 96-wells (250uL each)
4. Maintain sample integrity during redistribution
5. Use proper well mapping and tracking

T034 difficulty=Hard holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

Air bubble handling:

1. Transfer viscous liquids prone to bubble formation
2. Implement bubble detection techniques:
 - Visual inspection through aspiration pressure
 - Slow aspiration for bubble prevention
 - Dispense techniques to eliminate bubbles
3. Add bubble removal steps when detected
4. Include bubble handling in quality checks

T035 difficulty=Hard holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

Automated liquid level detection:

1. Set up wells with unknown liquid levels
2. Use liquid sensing to detect surface before aspiration
3. Handle cases where liquid level is too low
4. Adjust pipette depth automatically
5. Include fallback strategies for dry wells

T036 difficulty=Medium holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

Create a 10-fold serial dilution series across a 96-well plate:

- Starting concentration in A1: 10mg/mL
- Create 7 dilution points (A1 to A8)
- Each step should be 1:10 dilution
- Use buffer for dilutions
- Final volume in each well: 200uL
- Replicate each dilution in triplicate (rows A, B, C)

The robot should automatically calculate volumes and perform the dilutions.

T037 difficulty=Hard holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

DNA extraction from 24 bacterial samples using magnetic beads:

- Samples are in 1.5mL tubes in positions A1-D6 of tube rack
- Lyse cells with lysis buffer (200uL per sample)
- Add magnetic beads (50uL per sample)
- Mix and incubate 10 minutes
- Use magnetic module to separate beads
- Wash beads 3 times with wash buffer (500uL each wash)
- Elute DNA with elution buffer (100uL)
- Transfer final DNA to new clean tubes

Handle magnetic separation timing and bead resuspension properly.

T038 difficulty=Hard holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

Error handling and recovery test:

1. Set up a simple transfer protocol from tube to 96-well plate
2. Include error handling for:
 - Tip rack running out of tips (should pause and wait for new tip rack)
 - Insufficient liquid in source (should detect and skip/stop)
 - Well overflow detection (should prevent overfilling)
3. Add appropriate user notifications and logging
4. Implement recovery strategies where possible

T039 difficulty=Hard holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

Cell viability assay preparation up to off-platform reader handoff:

1. Seed cells in a 96-well plate
2. Add treatment compounds at specified concentrations
3. Incubate for 24 hours
4. Add viability detection reagents in the correct sequence
5. Mix on heater shaker 2 min at 500 rpm after each detection reagent
6. Prepare the plate for manual transfer to a plate reader
7. risk_checklist.json must declare off-platform handoff for fluorescence and luminescence reader measurements

Include proper controls, timing, and handoff notes.

T040 difficulty=Hard holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

Large volume reservoir management:

1. Set up large volume transfers from reservoir
2. Reservoir starts with 50mL total volume
3. Transfer 1mL aliquots to 48 wells (1mL each)
4. Track remaining volume
5. Handle reservoir depletion gracefully
6. Switch to backup reservoir if needed

T041 difficulty=Hard holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

Fluorescent assay plate preparation up to off-platform reader handoff:

1. Prepare assay wells with samples and controls
2. Add fluorescent reagent to each well
3. Mix gently and incubate 10 minutes at 37C in the dark
4. Wash 2x with HBSS buffer (200uL per well)

5. Prepare the plate for manual transfer to a fluorescence plate reader
 6. risk_checklist.json must declare off-platform handoff for fluorescence measurement
- Include proper timing, controls, and plate layout.

T042 difficulty=Hard holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

Multi-module coordination challenge:

1. Use magnetic module for DNA purification
2. Use temperature module for incubation steps
3. Use heater shaker for mixing and washing
4. Coordinate timing between modules:
 - Magnetic separation (5 min) -> Transfer -> Heating (37C, 10 min)
 - Mixing (500 rpm, 2 min) -> Cooling (4C, 5 min)

Include proper module initialization, timing control, and transfer between modules.

T043 difficulty=Hard holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

Pause and resume functionality:

1. Set up a multi-step protocol that can be paused:
 - Step 1: Add reagents to all wells
 - Step 2: Incubate 30 minutes (pause point)
 - Step 3: Add detection reagent
2. Use protocol.pause() for manual intervention points
3. Include resume instructions
4. Log pause/resume events
5. Handle protocol state preservation

T044 difficulty=Hard holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

Resource optimization:

1. Optimize for minimal tip usage across multiple reagents
2. Implement tip sharing strategy:
 - Group transfers by reagent type
 - Plan optimal tip change points
 - Calculate tip savings
3. Compare optimized vs non-optimized approaches
4. Include resource usage statistics

T045 difficulty=Medium holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

Simple drug addition workflow:

1. Dispense 200 uL of compound solution from 96-deepwell plate into assay plate
2. Add 50 uL of cell suspension to each well
3. Mix each well by pipetting up and down 3 times
4. Incubate 10 minutes
5. Add 20 uL of reagent solution and mix 2 times

Focus on basic liquid handling accuracy and timing.

T046 difficulty=Hard holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

Small volume precision challenge:

1. Use P10 pipette for precise small volumes
2. Transfer the following volumes accurately:
 - 1uL from source to destination
 - 2uL serial dilution (1:2, 1:4, 1:8)
 - 5uL mixing operations
3. Handle tip pre-wetting for accuracy
4. Include air gap to improve small volume handling

T047 difficulty=Expert holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

Pipette calibration preparation up to off-platform balance handoff:

1. Transfer known volumes into pre-labeled weigh vessels
2. Cover multiple volume ranges
3. Track target volume, vessel ID, and transfer order

4. Prepare vessels for manual weighing on a precision balance
5. risk_checklist.json must declare off-platform handoff for balance measurement and calibration analysis

Document the calibration workflow and handoff instructions.

T048 difficulty=Expert holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

Cell spheroid formation with osteoinduction (parameterized):

We are culturing cells (2,500 cells/100 ul) with DMEM in 6 well plates, and we want to make spheroids in 96 well plates under two conditions:

- 1) Control condition (OS-): 100 ul medium (DMEM) each well
- 2) Induction condition (OS+): 100 ul medium + supplements:
 - Dexamethasone: 10uL of 10uM stock (final 1uM)
 - Ascorbic acid: 10uL of 100uM stock (final 10uM)
 - Beta-glycerophosphate: 10uL of 100mM stock (final 10mM)

Add supplements to each well, then transfer cells to each well.

Split the code into multiple functions for each major step.

T049 difficulty=Expert holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

ELISA Assay Protocol:

1. Coat 96-well plate with capture antibody (100uL per well, 1ug/mL)
2. Incubate at room temperature for 2 hours
3. Wash plate 3 times with wash buffer (200uL per well)
4. Block with blocking buffer (150uL per well) for 1 hour
5. Wash 3 times again
6. Add samples in duplicate to wells A1-H6 (100uL per well)
7. Add standard curve samples to wells A7-H8 (8 points, duplicate)
8. Incubate 2 hours at room temperature
9. Wash 5 times
10. Add detection antibody (100uL per well, 0.5ug/mL)
11. Incubate 1 hour
12. Wash 5 times
13. Add substrate solution (100uL per well)
14. Incubate 15 minutes
15. Add stop solution (50uL per well)

Include proper reagent management and timing.

T050 difficulty=Hard holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

Emergency recovery procedures:

1. Implement protocol checkpoints:
 - Save progress after major steps
 - Enable resume from any checkpoint
 - Handle power failure recovery
2. Create data backup for all transfers
3. Include validation of recovered state
4. Test recovery scenarios

T051 difficulty=Hard holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

Environmental-control-aware plate preparation up to off-platform monitoring handoff:

1. Prepare cell-work plate transfers under stated environmental assumptions
 2. Add pause points for operator confirmation if temperature, humidity, or CO2 are out of range
 3. Log the expected environmental parameters in the runbook
 4. risk_checklist.json must declare off-platform handoff for environmental monitoring
- Include condition-based pause instructions rather than attempting to automate external environmental systems.

T052 difficulty=Expert holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

High-throughput drug screening plate preparation up to off-platform reader handoff:

1. Seed HeLa cells (5000 cells/well) in 384-well plate
2. Prepare drug dilution series: 8 concentrations x 48 compounds
3. Each compound tested in triplicate

4. Control wells: positive control (wells A1-A16), negative control (wells A17-A24)
 5. Add drugs using automated liquid-handling precision
 6. Add cell viability reagent (20uL per well) after the incubation step
 7. Mix by orbital shaking 2 minutes at 500 rpm, then prepare plate for manual reader transfer
 8. risk_checklist.json must declare off-platform handoff for luminescence measurement
- Automate liquid handling and plate layout up to the handoff point.

T053 difficulty=Expert holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

High-throughput dye-response calibration plate preparation up to off-platform reader handoff:

1. Set up a 384-well plate for dye calibration
 2. Prepare dye serial dilutions: 8 concentrations x 12 formulations
 3. Each formulation tested in triplicate
 4. Control wells: positive control (wells A1-A16, high dye), negative control (wells A17-A24, water)
 5. Add dye dilutions using automated liquid-handling precision
 6. Equilibrate 10 minutes
 7. Add neutral buffer (20 uL per well)
 8. Mix by orbital shaking 2 minutes at 500 rpm, then prepare plate for manual reader transfer
 9. risk_checklist.json must declare off-platform handoff for absorbance measurement
- Automate liquid handling and plate layout up to the handoff point.

T054 difficulty=Expert holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

Multi-protocol scheduling:

1. Schedule multiple protocols to run sequentially:
 - Protocol A: Sample preparation (30 min)
 - Protocol B: Assay setup (20 min)
 - Protocol C: Detection (15 min)
2. Handle resource sharing between protocols
3. Include buffer time between protocols
4. Generate complete schedule timeline

T055 difficulty=Expert holdout=no source=legacy_55 contract=execution_package

Can you write a Python script that does the following experiment?

RNA extraction from HEK293T cells using TRIzol:

1. Cells are in 6-well plate (90% confluence)
 2. Add 1mL TRIzol to each well, mix 3-4 times
 3. Wait 5 minutes for complete lysis
 4. Transfer lysate to 1.5mL tubes
 5. Add 200uL chloroform, mix, wait 3 minutes
 6. Centrifuge 15 minutes (manual step - add delay)
 7. Transfer aqueous phase (400uL) to new tubes
 8. Add 400uL isopropanol, mix, incubate 10 minutes
 9. Centrifuge 10 minutes, wash pellet with 75% ethanol
 10. Air-dry 5 minutes, resuspend in 30uL RNase-free water
- Include proper timing and manual step indications.

T056 difficulty=Easy holdout=yes source=new_35 contract=execution_package

default_samples=8 | expected_risk_flags=pipette mount compatibility; one tip per sample

Can you write a Python script that does the following experiment?

Basic 8-sample plate transfer on a single-arm benchtop liquid-handling workstation:

1. Load a 96-well destination plate in slot 1 and a tube rack with transfer_buffer in slot 2
 2. Mount a single-channel pipette and declare left or right mount explicitly so it matches the available rack
 3. Transfer 50 uL of transfer_buffer from the source tube into wells A1-A8 of the destination plate
 4. Use one fresh tip per sample (no tip reuse) and drop tips into the trash after each transfer
 5. Document the chosen mount and the one-tip-per-sample policy in the runbook
- Produce the complete seven-file execution package: a main script named `protocol.py` plus the six supporting files required by the execution-package schema.

T057 difficulty=Easy holdout=yes source=new_35 contract=execution_package
default_samples=24 | expected_risk_flags=runtime parameter bounds; sample count changes reagent and tip totals
Can you write a Python script that does the following experiment?
Runtime-parameterized dilution with selectable sample count:
1. Expose a runtime parameter sample_count that only accepts the values 8, 16, or 24
2. Load a 96-well plate in slot 1 and a reservoir of sample_diluent in slot 2
3. Transfer 25 uL of sample_diluent into sample_count destination wells (column-major from A 1)
4. Recompute reagent_plan.json totals and tip_plan.json counts from sample_count so they stay consistent
5. Reject the run with a clear error if sample_count is not in {8, 16, 24}
Produce the seven-file execution package and surface runtime parameter bounds in risk_checklist.json.

T058 difficulty=Medium holdout=yes source=new_35 contract=execution_package
default_samples=90 | expected_risk_flags=tip budget near rack limit; pause or extra rack policy
Can you write a Python script that does the following experiment?
Tip budget near 96-rack limit for 90 sample transfers:
1. Load the source plate in slot 2 and the destination plate in slot 1
2. Mount a single-channel pipette suitable for 20 uL transfers
3. Transfer 20 uL from source to destination for all 90 samples, one fresh tip per sample
4. Estimate total tip usage (90 sample tips plus any wash or priming tips) BEFORE the run and compare against a single 96-tip rack
5. Choose one of (a) loading a second rack, (b) pausing for operator tip refill, or (c) reusing tips, and justify the choice in the runbook
6. Record the chosen policy and the remaining-tip safety margin in risk_checklist.json
Produce the seven-file execution package and surface "tip budget near rack limit" in the risk checklist.

T059 difficulty=Medium holdout=yes source=new_35 contract=execution_package
default_samples=96 | expected_risk_flags=reservoir dead volume; source volume sufficiency
Can you write a Python script that does the following experiment?
Reagent distribution from a reservoir with dead-volume guard:
1. Load a 96-well destination plate in slot 1 and a 12-channel reservoir of reagent_buffer in slot 2
2. Mount an 8-channel pipette appropriate for 30 uL transfers
3. Distribute 30 uL of reagent_buffer to all 96 destination wells using by-column dispenses
4. Compute total required reagent (96 x 30 uL) plus per-channel dead volume, and verify the source volume is sufficient
5. Fail loudly if available source volume is less than required volume plus dead volume, and record the dead-volume margin in reagent_plan.json
Produce the seven-file execution package and surface reservoir dead volume and source sufficiency in risk_checklist.json.

T060 difficulty=Medium holdout=yes source=new_35 contract=execution_package
default_samples=48 | expected_risk_flags=multi-source allocation; source underfill avoidance
Can you write a Python script that does the following experiment?
Multi-source allocation of one reagent across 48 destinations:
1. Load a 96-well destination plate in slot 1 and a 24-tube rack in slot 2 with shared_reagent in tubes A1 and A3
2. Plan to transfer 15 uL of shared_reagent into each of 48 destination wells
3. Allocate destinations between source tubes A1 and A3 so neither tube drops below its minimum aspiration volume
4. Switch to the next source tube only when the current tube would underfill on the next aspirate, and log the switch event
5. Record which destinations come from which source tube in reagent_plan.json
Produce the seven-file execution package and surface multi-source allocation and underfill avoidance in risk_checklist.json.

T061 difficulty=Medium holdout=yes source=new_35 contract=execution_package
default_samples=24 | expected_risk_flags=deck slot conflict; module clearance
Can you write a Python script that does the following experiment?
Constrained deck layout with a module, reservoir, plate, tip racks, and trash:
1. Place a thermocycler or magnetic module in its supported slot and respect its clearance to neighboring slots
2. Load a 96-well plate (24 active wells), a reservoir of assay_reagent in slot 2, at least

one tip rack, and the trash

3. Verify no two labware or modules occupy the same slot and that the module's adjacent-slot restrictions are honored
 4. Transfer 40 uL of assay_reagent into 24 destination wells
 5. If a feasible layout does not exist, abort early with a clear deck-conflict error in the manifest instead of attempting to run
- Produce the seven-file execution package and surface deck slot conflict and module clearance in risk_checklist.json.

T062 difficulty=Medium holdout=yes source=new_35 contract=execution_package
default_samples=96 | expected_risk_flags=liquid waste capacity; tip disposal capacity
Can you write a Python script that does the following experiment?
High-volume wash for 96 wells with waste capacity planning:

1. Load a 96-well plate in slot 1 and a reservoir of wash_buffer in slot 2
 2. Dispense 600 uL of wash_buffer per well, then aspirate the same volume into a liquid waste container
 3. Compute total liquid waste volume (about 57.6 mL for 96 x 600 uL) and confirm the waste container capacity is sufficient
 4. Estimate total tips used and confirm the trash can hold them without overflow, otherwise plan a tip-disposal pause
 5. Document the waste container max fill, tip disposal capacity, and any planned pauses in the runbook
- Produce the seven-file execution package and surface liquid waste capacity and tip disposal capacity in risk_checklist.json.

T063 difficulty=Medium holdout=yes source=new_35 contract=execution_package
default_samples=24 | expected_risk_flags=high-to-low contamination order; fresh tip between concentration classes
Can you write a Python script that does the following experiment?
Mixed-concentration dilution with contamination-safe ordering:

1. Load the source plate in slot 3 (concentrated_sample) and a reservoir of dilution_buffer in slot 2
 2. Mark each of the 24 destinations as LOW or HIGH concentration and process LOW destinations first, HIGH last
 3. Place the positive control in A1 and the negative control in H12
 4. For each destination, pre-fill 80 uL dilution_buffer, then add 20 uL concentrated_sample with a fresh tip
 5. Use a fresh tip across every concentration-class boundary and never touch the dilution_buffer source with a tip that has already entered a sample well
- Produce the seven-file execution package and surface high-to-low contamination order and fresh tip between classes in risk_checklist.json.

T064 difficulty=Medium holdout=yes source=new_35 contract=execution_package
default_samples=24 | expected_risk_flags=control placement declared; template contamination prevention
Can you write a Python script that does the following experiment?
qPCR plate setup with positive, negative, and no-template controls:

1. Load a 96-well qPCR plate in slot 1, a tube rack with qpcr_master_mix in slot 2, and a template plate in slot 3
 2. Reserve A1 and A2 for positive controls, H12 for the negative control, and H11 for the no-template control (NTC)
 3. Dispense 10 uL qpcr_master_mix into 24 sample wells plus the four control wells
 4. Add 5 uL template into samples and positive controls; add 5 uL water into NTC; add nothing to the negative control
 5. Use a fresh tip for every template and control transfer, and process the control wells last so template aerosols cannot reach them first
- Produce the seven-file execution package, embed the control map in deck_plan.json, and surface control placement and template contamination prevention in risk_checklist.json.

T065 difficulty=Medium holdout=yes source=new_35 contract=execution_package
default_samples=24 | expected_risk_flags=master mix source protection; template added after master mix
Can you write a Python script that does the following experiment?
PCR master mix dispensing that protects the master mix source from template contamination:

1. Load a 96-well reaction plate in slot 1, a tube rack with pcr_master_mix in slot 2, and a sample plate of template in slot 3
2. Dispense 15 uL pcr_master_mix into 24 sample wells, the positive control (A1), the negative control (H12), and the NTC (H11), using fresh tips
3. Never reuse a master-mix tip after it has touched a sample or template well

4. Only after master mix is loaded, add 5 uL template into sample wells and the positive control; add water into NTC; leave the negative control empty
5. Seal or cap the master mix tube after dispensing if the runbook supports it
Produce the seven-file execution package and surface master mix source protection and template-added-after-master-mix in risk_checklist.json.

T066 difficulty=Medium holdout=yes source=new_35 contract=execution_package
default_samples=24 | expected_risk_flags=post-PCR contamination boundary; workspace separation
Can you write a Python script that does the following experiment?
Pre-PCR setup with an explicit post-PCR contamination boundary:
1. Load a 96-well reaction plate in slot 1, pre_pcr_master_mix on a tube rack in slot 2, and the template plate in slot 3
2. Perform only the pre-PCR steps on the robot: dispense 15 uL pre_pcr_master_mix and add 5 uL template per well
3. Place positive (A1), negative (H12), and NTC (H11) controls before thermocycling
4. Do NOT include any post-amplification step (gel loading, amplicon transfer, dilution of product) in protocol.py
5. In the runbook, declare a hard pre/post-PCR workspace boundary and list which steps must happen on a separate post-PCR bench
Produce the seven-file execution package and surface post-PCR contamination boundary and workspace separation in risk_checklist.json.

T067 difficulty=Medium holdout=yes source=new_35 contract=execution_package
default_samples=12 | expected_risk_flags=serial dilution carryover; mixing strategy consistency
Can you write a Python script that does the following experiment?
Serial dilution with a carryover-aware tip and mixing strategy:
1. Load a destination strip/plate in slot 1, dilution_buffer reservoir in slot 2, and stock_solution tube rack in slot 3
2. Pre-fill 12 dilution wells with 90 uL of dilution_buffer each (positive control at A1, negative at H12)
3. Transfer 10 uL from stock into the first well, mix 5 times by aspirate/dispense, then carry 10 uL forward to the next well
4. Use a fresh tip for every dilution step and every mix step; never reuse a tip across two dilution stages
5. Document the mixing volume, mixing count, and tip-per-step rule consistently in protocol.py and the runbook
Produce the seven-file execution package and surface serial dilution carryover and mixing strategy consistency in risk_checklist.json.

T068 difficulty=Medium holdout=no source=new_35 contract=execution_package
default_samples=24 | expected_risk_flags=enzyme warm time; small volume accuracy
Can you write a Python script that does the following experiment?
Enzyme addition with minimized warm time and small-volume accuracy:
1. Load the reaction plate in slot 1, reaction_buffer tube rack in slot 2, and enzyme_mix on a cold block (also slot 2 cold position)
2. Dispense 18 uL reaction_buffer into 24 sample wells plus the positive (A1) and negative (H12) controls FIRST
3. Aspirate enzyme_mix only after buffer is in place; transfer 2 uL of enzyme_mix per well using a low-volume tip with slow aspirate and dispense
4. Return each enzyme tube to the cold block immediately after the per-tube aliquot is finished to minimize warm time
5. Record per-tube cold-block time-out in the runbook so the operator can plan thaw and freeze cycles
Produce the seven-file execution package and surface enzyme warm time and small volume accuracy in risk_checklist.json.

T069 difficulty=Medium holdout=no source=new_35 contract=execution_package
default_samples=24 | expected_risk_flags=normalization input QC; replicate traceability
Can you write a Python script that does the following experiment?
DNA normalization with replicate traceability and input QC:
1. Read per-sample input concentration from sample_plate_slot_3 metadata and validate that every value falls in a usable range
2. For each of 24 samples, compute the dna_sample volume (5 uL nominal here) and normalization_buffer volume (45 uL nominal) needed to hit the target concentration
3. Transfer dna_sample first, then normalization_buffer, using a fresh tip per sample (including positive control A1 and negative control H12)

4. Mix each destination well after both additions and record final volume plus target concentration in reagent_plan.json

5. Tag each destination with its originating sample ID so replicates remain traceable in the manifest

Produce the seven-file execution package and surface normalization input QC and replicate traceability in risk_checklist.json.

T070 difficulty=Hard holdout=yes source=new_35 contract=execution_package

default_samples=96 | expected_risk_flags=PCR setup contamination; thermocycler handoff

Can you write a Python script that does the following experiment?

Full 96-well PCR setup with master mix, primers, and template:

1. Load a 96-well PCR plate in slot 1, a tube rack with pcr_master_mix and primer_mix in slot 2, and a template plate in slot 3

2. Dispense 12.5 uL pcr_master_mix into all 96 wells, then add 2.5 uL primer_mix into each well using fresh tips

3. Reserve A1 for the positive control, H12 for the negative control, and H11 for the NTC; add 5 uL template into samples and the positive control, water into NTC, and nothing into the negative control

4. Finish master-mix and primer additions BEFORE any template tube is opened to keep the master mix source clean

5. Document the thermocycler handoff in the runbook: target program, lid temperature, expected run time, and that the operator moves the sealed plate to the cycler

Produce the seven-file execution package and surface PCR setup contamination and thermocycler handoff in risk_checklist.json.

T071 difficulty=Hard holdout=yes source=new_35 contract=execution_package

default_samples=24 | expected_risk_flags=triplicate layout consistency; qPCR contamination control

Can you write a Python script that does the following experiment?

qPCR plate setup with standards, triplicate samples, and double NTC:

1. Load the qPCR plate in slot 1, qpcr_master_mix on a cold block in slot 2, and the standards/template plate in slot 3

2. Lay out 24 samples in triplicate (8 unique samples x 3) plus standards; positive controls at A1-A3, negative at H12, NTCs at H10 and H11

3. Dispense 10 uL qpcr_master_mix into every used well, then add 5 uL standard_or_template into samples, standards, and positive controls

4. Maintain a consistent column-major (or row-major) triplicate layout so replicates of one sample share a column or row and are never interleaved with other samples

5. Use a fresh tip per well and never aspirate template above the master mix tube

Produce the seven-file execution package and surface triplicate layout consistency and qPCR contamination control in risk_checklist.json.

T072 difficulty=Hard holdout=yes source=new_35 contract=execution_package

default_samples=96 | expected_risk_flags=wash residual volume; plate handling during repeated washes

Can you write a Python script that does the following experiment?

ELISA add-and-wash with repeated wash cycles for 96 wells:

1. Load the coated 96-well plate in slot 1, a wash_buffer reservoir in slot 2, and a detection_reagent reservoir in slot 3

2. Run three wash cycles per stage: dispense 1000 uL wash_buffer per well, then aspirate into a liquid waste container, leaving less than 10 uL residual

3. After the final wash, blot or tap the plate per the runbook instructions to remove residual wash before reagent addition

4. Add 100 uL detection_reagent to all 96 wells, with positive controls at A1 and A2 and negative controls at H11 and H12

5. Track plate orientation across every transfer step so column 1 stays column 1 and no wells are skipped

Produce the seven-file execution package and surface wash residual volume and plate handling during repeated washes in risk_checklist.json.

T073 difficulty=Hard holdout=yes source=new_35 contract=execution_package

default_samples=16 | expected_risk_flags=cold chain for assembly mix; molar ratio feasibility

Can you write a Python script that does the following experiment?

Gibson assembly setup with cold-chain master mix and molar-ratio sanity check:

1. Load the reaction strip or plate in slot 1, gibson_master_mix on a cold block in slot 2, and vector_insert_mix tube rack in slot 3

2. Compute the vector-to-insert molar ratio for each of the 16 reactions from input concentrations; fail loudly if the ratio falls outside the recommended 1:2 to 1:5 window

3. Transfer 10 uL vector_insert_mix into each reaction well, plus positive (A1) and negative (H12) controls

4. Aspirate gibbon_master_mix from the cold block, dispense 10 uL per well, then return the tube to the cold block immediately

5. Track total time the master mix tube is outside the cold block and abort or warn if it exceeds the documented threshold

Produce the seven-file execution package and surface cold chain for assembly mix and molar ratio feasibility in risk_checklist.json.

T074 difficulty=Hard holdout=yes source=new_35 contract=execution_package default_samples=16 | expected_risk_flags=enzyme warm time; assembly control placement

Can you write a Python script that does the following experiment?

Golden Gate assembly setup with controlled enzyme handling:

1. Load the reaction plate in slot 1, golden_gate_master_mix on a cold block in slot 2, and acceptor_insert_mix tube rack in slot 3
2. Dispense 15 uL acceptor_insert_mix into 16 reaction wells, plus positive (A1) and negative (H12) controls
3. Pipette 5 uL golden_gate_master_mix per well from the cold block, returning enzyme tubes to the block between every aspirate
4. Verify that control wells receive the matching no-insert / no-enzyme combinations expected for Golden Gate validation
5. Document the thermocycler program (digestion plus ligation cycles) and that the operator moves the sealed plate to the cycler

Produce the seven-file execution package and surface enzyme warm time and assembly control placement in risk_checklist.json.

T075 difficulty=Hard holdout=yes source=new_35 contract=execution_package default_samples=24 | expected_risk_flags=manual colony picking handoff; template carryover

Can you write a Python script that does the following experiment?

Colony PCR setup with a manual colony-picking handoff:

1. Load a PCR plate in slot 1, colony_pcr_master_mix on a cold block in slot 2, and a colony_template_suspension plate (prepared manually) in slot 3
2. Dispense 18 uL colony_pcr_master_mix into 24 sample wells, the positive control (A1), the negative control (H12), and the NTC (H11)
3. Insert an operator pause in the scripted run with a message: "Pick colonies into source wells A1..A24 of slot 3 and then resume."
4. After resume, transfer 2 uL colony_template_suspension into the matching reaction well using a fresh tip per colony to avoid template carryover
5. Add 2 uL water into NTC and leave the negative control empty of template

Produce the seven-file execution package and surface manual colony picking handoff and template carryover in risk_checklist.json.

T076 difficulty=Hard holdout=yes source=new_35 contract=execution_package default_samples=24 | expected_risk_flags=restriction enzyme cold handling; small enzyme volume accuracy

Can you write a Python script that does the following experiment?

Restriction digest setup with cold-handled enzyme and small DNA volumes:

1. Load the reaction plate in slot 1, restriction_digest_mix on a cold block in slot 2, and dna_sample plate in slot 3
2. Dispense 18 uL restriction_digest_mix into 24 reactions plus the positive (A1) and negative (H12) controls
3. Add 2 uL dna_sample per reaction using a low-volume pipette and matching tips (for example 0.5–20 uL class) with slow aspirate and reverse pipetting where supported
4. Keep restriction_digest_mix on the cold block at all times except during aspiration; record total warm time per tube
5. Mix each reaction gently (no enzyme bubbling) and seal before moving to the incubator

Produce the seven-file execution package and surface restriction enzyme cold handling and small enzyme volume accuracy in risk_checklist.json.

T077 difficulty=Hard holdout=no source=new_35 contract=execution_package default_samples=24 | expected_risk_flags=magnetic separation handoff; ethanol residual carryover

Can you write a Python script that does the following experiment?

NGS library cleanup bead substep with magnetic separation handoff:

1. Load the sample plate in slot 1, magnetic_beads reservoir in slot 2, and ethanol_80_percent reservoir in slot 3
2. Add 45 uL magnetic_beads to each of 24 sample wells; mix 10 times by aspirate/dispense; incubate at room temperature for the documented time

3. Pause and prompt the operator to move the plate onto the magnetic module; resume only after a confirmed magnet-engaged state

4. Aspirate and discard the supernatant; perform two 400 uL ethanol_80_percent washes WITHOUT disturbing the bead pellet

5. Air-dry the pellet for the documented time and call out residual ethanol risk in both the runbook and risk_checklist.json

Produce the seven-file execution package and surface magnetic separation handoff and ethanol residual carryover in risk_checklist.json.

T078 difficulty=Hard holdout=no source=new_35 contract=execution_package default_samples=24 | expected_risk_flags=concentration input validation; minimum transferable volume

Can you write a Python script that does the following experiment?

Plasmid DNA normalization from concentration to target mass and final volume:

1. Read per-sample plasmid concentration (ng/uL) from sample_plate_slot_3 metadata
2. For each of 24 samples, compute plasmid_sample volume and nuclease_free_water volume needed to hit a target mass at a target final volume (for example 200 ng in 50 uL)
3. If the required plasmid volume falls below the pipette's minimum transferable volume, flag the sample as infeasible and SKIP it with a clear note instead of pipetting an unreliable amount
4. Transfer water first, then plasmid; mix each well gently; reserve positive control at A1 and negative at H12
5. Record per-sample input concentration, computed volumes, and any skipped samples in reagent_plan.json

Produce the seven-file execution package and surface concentration input validation and minimum transferable volume in risk_checklist.json.

T079 difficulty=Hard holdout=yes source=new_35 contract=execution_package default_samples=24 | expected_risk_flags=CSV schema validation; sample ID traceability

Can you write a Python script that does the following experiment?

CSV-driven sample sheet processing with traceable plate mapping:

1. Read a CSV sample sheet (uploaded as a runtime CSV parameter) and validate the columns: sample_id, source_well, destination_well, target_volume_uL
2. Reject the run if any required column is missing, any sample_id is duplicated, or any well address is malformed
3. Load the source plate in slot 3 and a reservoir of diluent in slot 2
4. For each row, transfer 20 uL sample_from_csv from the mapped source well to the destination well, then top up with 30 uL diluent (positive control at A1, negative at H12)
5. Write the full sample_id-to-destination_well mapping into manifest.json so the run is reproducible and traceable

Produce the seven-file execution package and surface CSV schema validation and sample ID traceability in risk_checklist.json.

T080 difficulty=Hard holdout=yes source=new_35 contract=execution_package default_samples=192 | expected_risk_flags=plate format selection threshold; well mapping consistency

Can you write a Python script that does the following experiment?

Plate format selection based on sample count (96 vs 384):

1. Read a runtime parameter sample_count and choose the destination plate: less than or equal to 96 -> 96-well plate, greater than 96 -> 384-well plate (here 192 -> 384-well)
2. Load the chosen plate plus a reservoir of assay_reagent in slot 2; deck_plan.json must reflect the chosen plate, not both
3. Generate a deterministic column-major well map for sample_count destinations and document the chosen format and well map in the runbook
4. Dispense 10 uL assay_reagent into the 192 destinations on the 384-well plate; positive control at A1, negative at P24
5. Keep deck_plan.json, reagent_plan.json, tip_plan.json, and runbook.md consistent with the selected plate format (no stale 96-well references when 384-well is chosen)

Produce the seven-file execution package and surface plate format selection threshold and well mapping consistency in risk_checklist.json.

T081 difficulty=Hard holdout=yes source=new_35 contract=execution_package default_samples=24 | expected_risk_flags=pipette mount availability; volume range compatibility

Can you write a Python script that does the following experiment?

Mount-aware pipette selection with volume-range guards:

1. Accept runtime parameters pipette_mount in {left, right} and pipette_model
2. Verify the requested pipette can deliver 75 uL transfer_solution within its supported volume range; fail with a clear error if not

3. Verify the chosen mount is free (no module or labware blocking the head travel) and that the loaded tip rack matches the pipette type
 4. Distribute 75 uL transfer_solution from a reservoir in slot 2 into 24 destination wells with one tip per sample
 5. Document the chosen mount, pipette model, and tip rack pairing in manifest.json
- Produce the seven-file execution package and surface pipette mount availability and volume range compatibility in risk_checklist.json.

T082 difficulty=Hard holdout=yes source=new_35 contract=execution_package

default_samples=48 | expected_risk_flags=primary source insufficiency; backup source policy

Can you write a Python script that does the following experiment?

Backup-source policy when the primary reagent tube runs low:

1. Load a 24-tube rack in slot 2 with primary_reagent_source in A1 and backup_reagent_source in A2; destination plate in slot 1
2. Plan 48 transfers of 25 uL each, drawing from primary_reagent_source first
3. Before each aspirate, check tracked remaining volume in A1; if A1 would drop below its minimum aspiration volume, switch to A2 BEFORE that transfer (not after a failed aspirate)
4. Record the exact destination at which the source switched, in reagent_plan.json
5. If A2 also cannot satisfy the remaining transfers, abort with a clear error rather than producing partial results

Produce the seven-file execution package and surface primary source insufficiency and backup source policy in risk_checklist.json.

T083 difficulty=Hard holdout=yes source=new_35 contract=execution_package

default_samples=24 | expected_risk_flags=module state confirmation; unsafe module interaction pause

Can you write a Python script that does the following experiment?

Module-aware assay with state confirmation and operator pause:

1. Load a module (thermocycler, magnetic, or temperature module) in its supported slot and the assay plate in slot 1
2. Document the expected module state (lid open, target temperature reached, magnet disengaged) before pipetting begins
3. Use operator pauses (surfaced messages that halt until acknowledged) to confirm module state whenever the module's state cannot be polled programmatically
4. Transfer 20 uL module_assay_reagent into 24 wells with positive control at A1 and negative control at H12
5. Forbid any pipette move into the module's well area while the module is in an unsafe state (lid closed, magnet engaged, plate too hot)

Produce the seven-file execution package and surface module state confirmation and unsafe module interaction pause in risk_checklist.json.

T084 difficulty=Hard holdout=no source=new_35 contract=execution_package

default_samples=24 | expected_risk_flags=replicate count layout update; replicate count reagent scaling

Can you write a Python script that does the following experiment?

Replicate-count parameter that consistently scales layout, reagent, and tips:

1. Expose a runtime parameter replicate_count in {1, 2, 3, 4}; default is 1 with 24 samples giving 24 destinations
2. Compute total destinations as 24 * replicate_count and lay them out in column-major order on a 96-well plate; reject if total exceeds 96
3. Transfer 10 uL replicate_sample per destination from sample_plate_slot_3, then 40 uL assay_buffer from reservoir_slot_2
4. Update reagent_plan.json totals and tip_plan.json counts to match replicate_count automatically (no manual editing required)
5. Place positive control at A1 and negative at H12 within every replicate block and record replicate_count in manifest.json

Produce the seven-file execution package and surface replicate count layout update and replicate count reagent scaling in risk_checklist.json.

T085 difficulty=Hard holdout=no source=new_35 contract=execution_package

default_samples=24 | expected_risk_flags=conditional control manifest alignment; optional reagent volume scaling

Can you write a Python script that does the following experiment?

Conditional controls and optional reagent that stay aligned across all package files:

1. Expose a runtime parameter include_controls (bool). When true, place positive (A1), negative (H12), and NTC (H11) and load optional_control_reagent on a tube rack in slot 2
2. When include_controls is false, drop the control wells entirely and do NOT load optional_control_reagent
3. Transfer 20 uL sample_reagent from sample_plate_slot_3 into all 24 sample wells in both m

odes

4. When include_controls is true, also transfer 20 uL optional_control_reagent into the three control wells using fresh tips
 5. Make sure deck_plan.json, reagent_plan.json, tip_plan.json, runbook.md, and risk_checklist.json all reflect the chosen mode (no stale control entries when controls are off)
- Produce the seven-file execution package and surface conditional control manifest alignment and optional reagent volume scaling in risk_checklist.json.

T086 difficulty=Medium holdout=yes source=new_35 contract=execution_package

default_samples=8 | expected_risk_flags=seven file consistency; operator review readiness

Can you write a Python script that does the following experiment?

Seven-file package consistency for a simple 8-sample transfer (review readiness):

1. Load a destination plate in slot 1 and a reservoir of transfer_buffer in slot 2
2. Transfer 50 uL transfer_buffer into 8 destination wells with one fresh tip per sample
3. Emit ALL seven files: main script (name it `protocol.py` in the package), deck_plan.json, reagent_plan.json, tip_plan.json, runbook.md, risk_checklist.json, manifest.json
4. Cross-check that slot numbers, labware registry names, well lists, reagent volumes, and tip counts in the main script match the JSON files and the runbook narrative exactly
5. Make the runbook readable end-to-end by a human reviewer who has never opened the main script file

Produce the seven-file execution package and surface seven file consistency and operator review readiness in risk_checklist.json.

T087 difficulty=Medium holdout=yes source=new_35 contract=execution_package

default_samples=24 | expected_risk_flags=simulator-checkable risk separation; human setup risk separation

Can you write a Python script that does the following experiment?

Risk checklist that separates simulator-checkable risks from human-setup risks:

1. Load a destination plate in slot 1 and reservoir of assay_reagent in slot 2; positive control at A1, negative at H12
2. Transfer 25 uL assay_reagent into 24 sample wells and the two controls with one tip per sample
3. In risk_checklist.json, group items under "simulator_checkable" (deck conflicts, volume math, tip budget) and "human_setup" (correct reagent in correct slot, plate seal, liquid color, refrigeration)
4. Each risk item must include: category, description, detection method, and required operation
5. Do not mix simulator-checkable and human-only risks in the same group, and do not duplicate items across groups

Produce the seven-file execution package and surface simulator-checkable risk separation and human setup risk separation in risk_checklist.json.

T088 difficulty=Medium holdout=yes source=new_35 contract=execution_package

default_samples=8 | expected_risk_flags=manifest reproducibility; retry budget recorded

Can you write a Python script that does the following experiment?

Reproducibility-grade manifest for a simple 8-sample transfer:

1. Load a destination plate in slot 1 and reservoir of transfer_buffer in slot 2
2. Transfer 50 uL transfer_buffer into 8 destination wells with one fresh tip per sample
3. In manifest.json record: model name, scaffold/template version, SHA-256 of the prompt, retry budget consumed and remaining, permissions/scopes used, and an ISO 8601 timestamp
4. Include the git commit (or "uncommitted") of the delivered package (or of `protocol.py`) so the run can be re-executed exactly
5. Keep the manifest field set stable across reruns of the same prompt so diffs between runs remain meaningful

Produce the seven-file execution package and surface manifest reproducibility and retry budget recorded in risk_checklist.json.

T089 difficulty=Medium holdout=no source=new_35 contract=execution_package

default_samples=24 | expected_risk_flags=operator setup completeness; manual check clarity

Can you write a Python script that does the following experiment?

Operator-facing runbook for deck setup, labware, and manual checks:

1. Load a destination plate in slot 1 and reservoir of setup_reagent in slot 2; positive control at A1, negative at H12
2. Transfer 30 uL setup_reagent into 24 sample wells and the two controls
3. Write runbook.md so an operator with NO access to the main script (`protocol.py`) can identify each labware by registry name and slot, fill each reagent to a stated minimum volume, pick the correct pipette and tip rack, and run a final pre-run checklist

4. Include a numbered manual-check list with explicit pass/fail criteria for each item (for example "tip rack count >= X" or "reservoir reagent color matches")

5. Cross-reference each manual check with the matching entry in risk_checklist.json

Produce the seven-file execution package and surface operator setup completeness and manual check clarity in risk_checklist.json.

T090 difficulty=Medium holdout=no source=new_35 contract=execution_package
default_samples=8 | expected_risk_flags=deck reagent inconsistency repaired; repair rationale documented

Can you write a Python script that does the following experiment?

Repair an inconsistent protocol package and document the fix:

1. Treat the input as a draft seven-file package where deck_plan.json says slot 2 holds a tube rack but reagent_plan.json says repair_test_reagent comes from a reservoir
2. Decide which file is authoritative based on the main script ('protocol.py') behavior and the runbook, then update the other files to match (do not silently rewrite 'protocol.py')
3. Plan an 8-sample transfer of 50 uL repair_test_reagent from the resolved slot 2 source in to destination plate (slot 1) wells A1-A8
4. Record the specific file diffs and the rationale for each repair decision in manifest.json under a "repairs" entry
5. Re-run the seven-file consistency check after the repair and refuse to ship if any inconsistency remains

Produce the seven-file execution package and surface deck reagent inconsistency repaired and repair rationale documented in risk_checklist.json.